

# TCP+simulink 测试流程说明



包含模型以及脚本代码（Python 3.12.7）

| 代码         | 功能                                                   |
|------------|------------------------------------------------------|
| Test.py    | 建立 Simulink、上层算法以及数据可视化界面的连接，实现车辆底层状态数据的采集和上层控制指令的下发 |
| Send.py    | 连接 7000 端口，并向系统下发实时更新的控制指令                           |
| Receive.py | 连接 6000 端口，将 底层数据转化为直观的实时动态折线图                       |

| 端口号     | 功能                                |
|---------|-----------------------------------|
| 端口 5000 | 指令下发端，上层更新的控制指令发送给 Simulink 模型去执行 |
| 端口 5001 | 数据采集端，接收来自 Simulink 实时发送的车辆状态数据   |
| 端口 6000 | 数据监控端，实时获取到从 5001 端口透传过来车辆状态数据    |
| 端口 7000 | 上层决策端，接收外部算法模块下发的最新决策指令           |

步骤 1：在三个终端中依次执行 Test.py Send.py Receive.py

步骤 2：运行成功结果如下示意图，此时运行仿真模型，可视化界面实时反应车辆状态

```
=====
正在启动 Python - Simulink 通讯节点...
=====
服务器已在端口 5000 启动（等待 Simulink 接收模块连接...）
服务器已在端口 5001 启动（等待 Simulink 发送模块连接...）
服务器已在端口 6000 启动（等待前端画面界面连接...）
服务器已在端口 7000 启动

所有节点已就绪。按 Ctrl+C 可随时退出。

[内部总线] 绘图界面 ('127.0.0.1', 56055) 已接入 6000 端口！

AI计算 ('127.0.0.1', 56063) 已接入 7000 端口！
[]

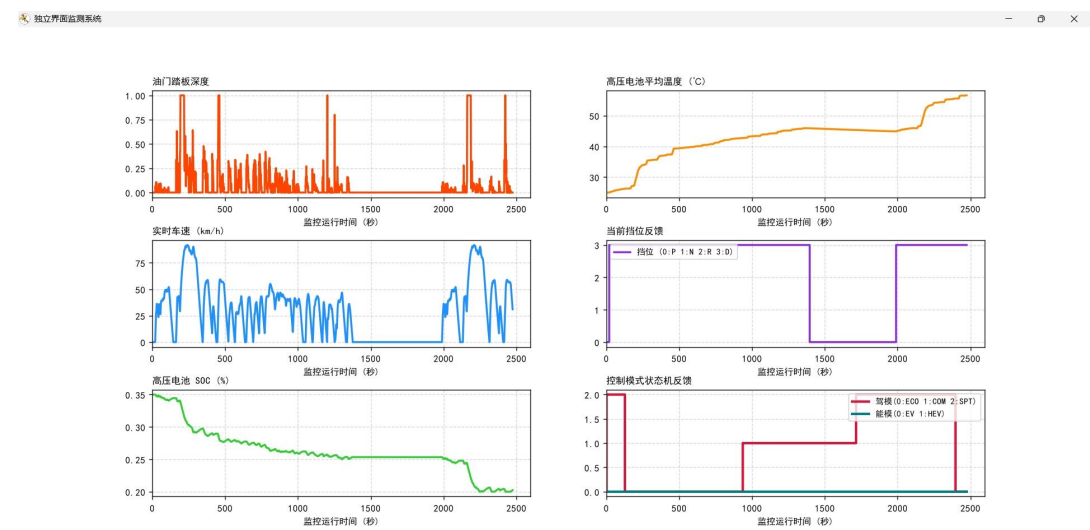
PS E:\matlab\STLA\python - 副本> python .\Receive.py
=====
监测大屏已启动
=====
正在尝试连接后端服务 (端口 6000)...
成功连接！开始接收全量数据。
findfont: Failed to find font weight bold, now using 400.
[]

PS E:\matlab\STLA\python - 副本> python .\Send.py
=====
AI已启动
=====
尝试连接指令网关 (TCP 7000)...
连接成功！算法控制权已接管。
[]
```

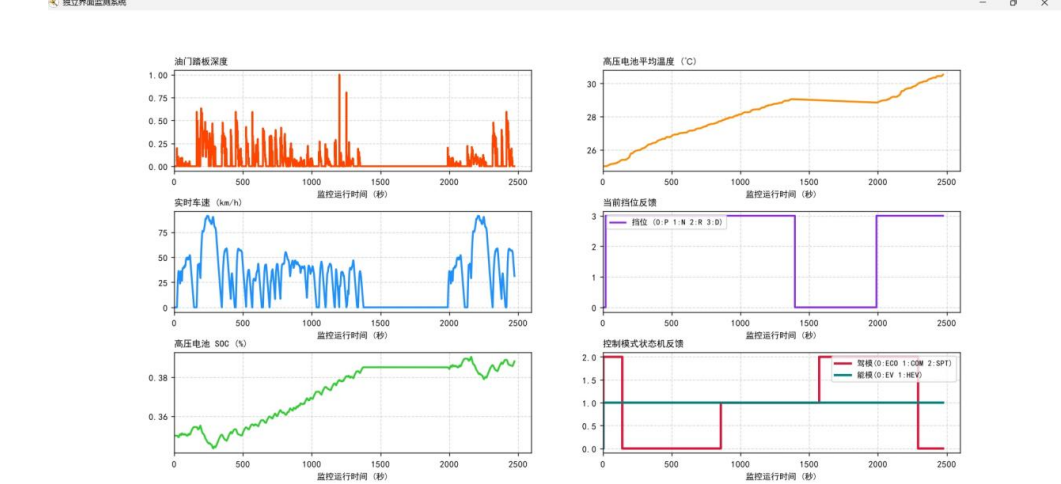
步骤 3：更改 Test.py 中 CASE\_CHOOSE 可进行测试案例选择（EV:纯电 HEV:混合 其他：EV+HEV）

```
Send.py
1 import socket
2 import threading
3 import struct
4 import time
5 CASE_CHOOSE = "HEV"
6 DRIVE_MODES = [0.0, 1.0, 2.0]
7
```

# 案例一：EV



# 案例二：HEV



# 案例三：HEV+EV

